

# AI-BFSI Internship Project Documentation

## Code Layout, Readability & Reusability – Subscription Waste Detector

### 1. Introduction

Well-structured source code is essential for high-quality software. Clean layout improves readability, simplifies debugging, reduces maintenance effort, and enables future enhancements.

In the **Subscription Waste Detector**, emphasis was placed on writing organized and maintainable Python code. The application was structured into logical sections for user interaction, data processing, subscription detection, visualization, and AI-powered recommendation generation. The goal was to produce software that is easy to understand, modify, and extend while maintaining consistent coding practices.

### 2. Objectives

- Improve source code readability
- Maintain logical program organization
- Reduce unnecessary duplication
- Simplify debugging and testing
- Encourage modular development
- Support future enhancements
- Improve long-term maintainability

### 3. Code Organization

The source code was organized by functional responsibility:

| Component                | Responsibility                                        |
|--------------------------|-------------------------------------------------------|
| Streamlit Interface      | User interaction and dashboard presentation           |
| Data Processing          | Reading and preparing uploaded expense data           |
| Subscription Detection   | Identifying recurring subscription transactions       |
| Data Visualization       | Generating interactive charts using Plotly            |
| AI Recommendation Module | Communicating with Groq API to obtain recommendations |

### 4. Readability Practices

- **Meaningful Variable Names:** e.g., `expense_data`, `subscription_list`, `monthly_spending`, `renewal_date`, `ai_recommendations`.
- **Consistent Formatting:** Uniform indentation, logical spacing, clear separation of functions, consistent bracket placement.

- **Inline Comments:** Used to explain complex logic (e.g., subscription detection), not to restate obvious code.
- **Logical Code Flow:** Input → Processing → Detection → Visualization → AI Recommendations → Output.

## 5. Modular Design

Functionality was grouped into independent modules:

- File upload handling
- Data preprocessing
- Subscription analysis
- Dashboard generation
- AI recommendation generation

This modular design improves maintainability and reduces dependencies.

## 6. Code Reusability

- **Common Data Processing:** Shared workflow for CSV and Excel uploads.
- **Shared Visualization Logic:** Reusable Plotly functions for charts.
- **AI Recommendation Module:** Groq API integration isolated for easy updates.
- **Consistent Pipeline:** Same detection workflow for uploaded files and manual entries.

## 7. Maintainability

- Clear separation of responsibilities
- Consistent naming conventions
- Modular implementation
- Readable program flow
- Reduced duplication

These practices simplify debugging and future feature additions.

## 8. Error Handling

Basic validation was implemented:

- File format validation
- Handling empty datasets
- Exception handling during data processing
- Validation before AI recommendation generation

## 9. Coding Standards Followed

| Practice          | Implementation |
|-------------------|----------------|
| Meaningful Naming | Implemented    |

|                              |             |
|------------------------------|-------------|
| <b>Modular Structure</b>     | Implemented |
| <b>Consistent Formatting</b> | Implemented |
| <b>Reduced Duplication</b>   | Implemented |
| <b>Logical Program Flow</b>  | Implemented |
| <b>Basic Error Handling</b>  | Implemented |
| <b>Readable Functions</b>    | Implemented |

## 10. Benefits of Code Structure

- **Improved Readability:** Easier to understand.
- **Easier Debugging:** Errors isolated by module.
- **Better Maintainability:** Changes localized.
- **Increased Reusability:** Modules reused for future features.
- **Simplified Testing:** Independent functional sections.

## 11. Opportunities for Future Improvement

Potential enhancements:

- Dedicated utility modules
- Configuration files for settings
- Automated unit testing
- Logging mechanisms
- Database abstraction layers
- API service abstraction
- Type annotations

## 12. Conclusion

The Subscription Waste Detector was developed using coding practices that emphasize readability, maintainability, and reusability. By organizing the application into logical components, following consistent naming conventions, reducing duplication, and maintaining a clear workflow, the project achieves a clean and structured implementation aligned with professional software engineering standards.